---------- Workshop #4 ----------

- This workshop includes marked tasks that comprise 15% of your final mark in this module.
- You need to read the examples in Lecture #4 and Lecture #4 exercise to complete the tasks.

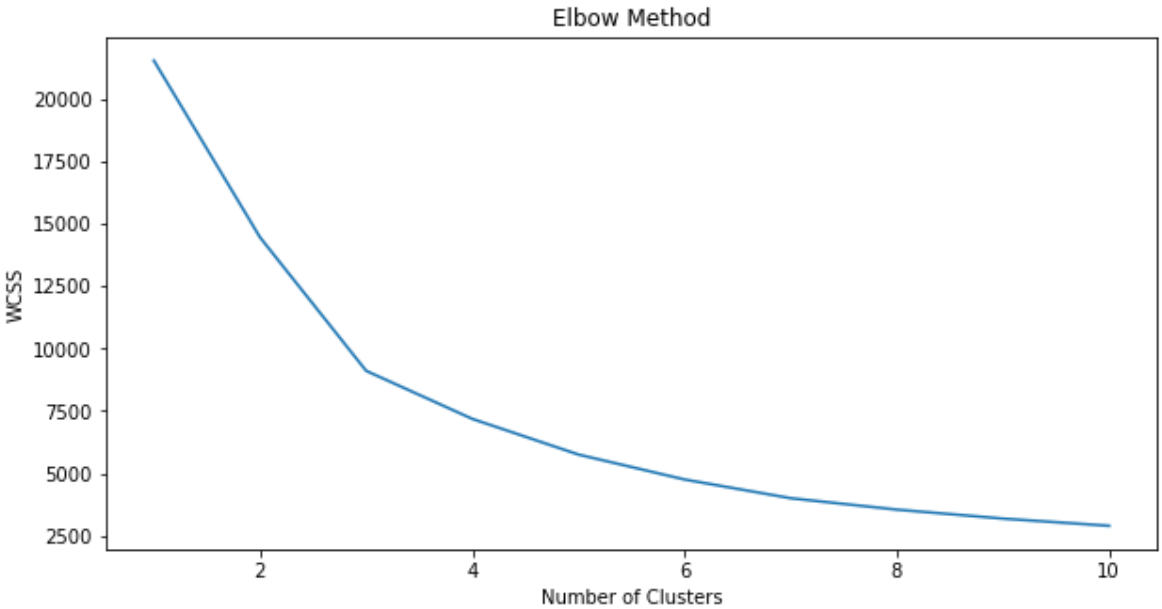
Tasks

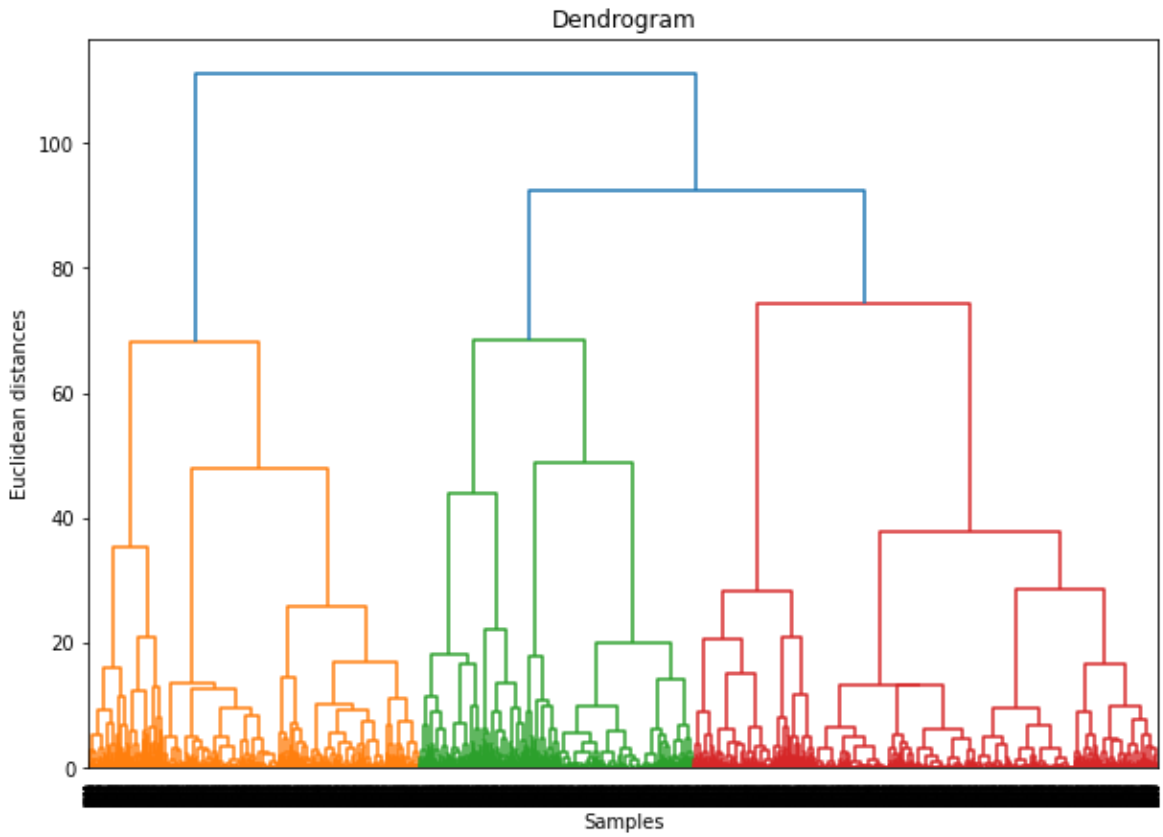
TASK 4.1: Download the adult_WS4 dataset. Apply K-Means and Hierarchical clustering to three optional columns in the dataset. Find the optimum number of clusters for both clustering methods (10%).

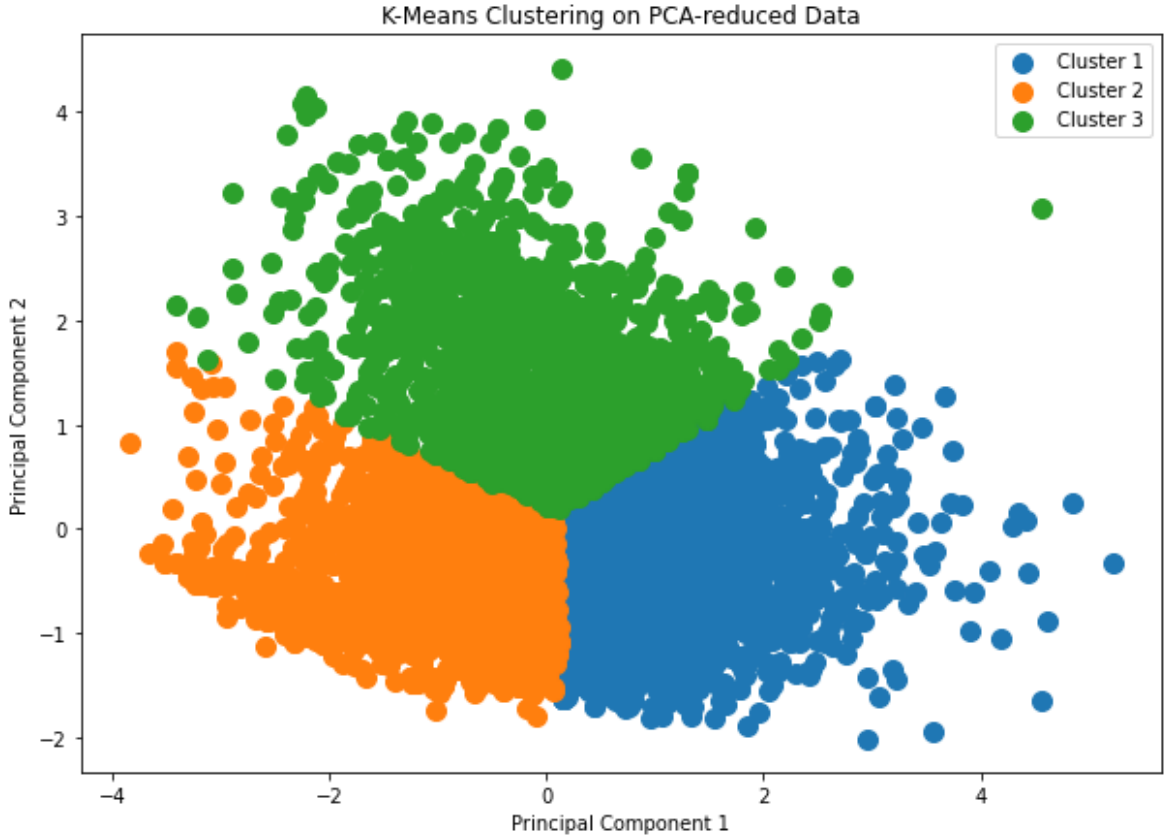
NOTE: You should comment on your code wherever necessary and briefly explain what the code is doing

In [28]:
```
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt
import scipy.cluster.hierarchy as sch

# Load dataset
data = pd.read_csv('https://raw.githubusercontent.com/maz786/WlvMScAI/mai

# Selecting columns and standardize to preprocess data
X = data[['age', 'education-num', 'hours-per-week']]
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Reduce dimensionality by applying PCA
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

# Finding optimal number of clusters using Elbow method for K-Means Clust
wcss = []
for i in range(1, 11):
```

```
    kmeans = KMeans(n_clusters=i, init='k-means++', random_state=42, n_in
    kmeans.fit(X_pca)
    wcss.append(kmeans.inertia_)

plt.figure(figsize=(10, 5))
plt.plot(range(1, 11), wcss)
plt.title('Elbow Method')
plt.xlabel('Number of Clusters')
plt.ylabel('WCSS')
plt.show()

# Choosing number of clusters based on elbow plot apply K-Means
optimal_clusters = 3
kmeans = KMeans(n_clusters=optimal_clusters, init='k-means++', random_sta
y_kmeans = kmeans.fit_predict(X_pca)

# Generate Dendrogram for Hierarchical Clustering
plt.figure(figsize=(10, 7))
dendrogram = sch.dendrogram(sch.linkage(X_scaled, method='ward'))
plt.title('Dendrogram')
plt.xlabel('Samples')
plt.ylabel('Euclidean distances')
plt.show()

# K-Means Clusters Visualization
plt.figure(figsize=(10, 7))
for i in range(optimal_clusters):
    plt.scatter(X_pca[y_kmeans == i, 0], X_pca[y_kmeans == i, 1], s=100,
plt.title('K-Means Clustering on PCA-reduced Data')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.legend()
plt.show()
```







############# WRITE YOUR REPORT IN THIS CELL (IF APPLICABLE)####################

The provided code uses pandas (McKinney, 2010), numpy (Van Der Walt, Colbert, & Varoquaux, 2011), scikit-learn (Pedregosa et al., 2011), and matplotlib (Hunter, 2007) to perform data preparation, dimensionality reduction, clustering, and visualisation on a dataset. It uses StandardScaler to standardise chosen data, PCA to reduce dimensions (Jolliffe, 2002), the Elbow Method to identify optimal clusters (Kodinariya & Makwana, 2013), K-Means clustering (MacQueen, 1967), and Johnson (1967) to

visualise hierarchical clustering dendrograms. These examples demonstrate how these libraries are integrated for efficient data analysis and machine learning implementation.

Task 4.2: Apply the PCA method to the dataset and extract the first two principal components (n_components=2). Plot the scatter plot of the dataset's first two components for the two classes of the income column (5%).

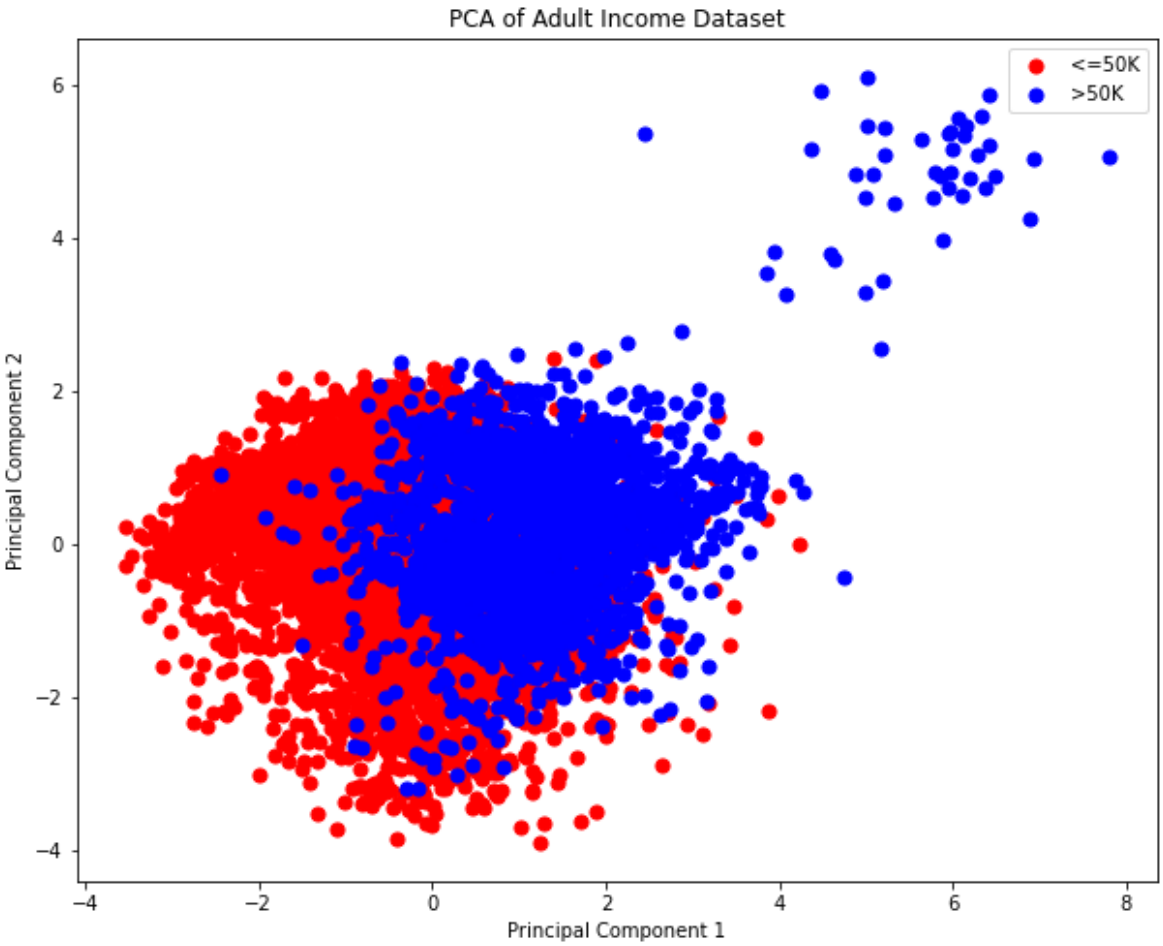
NOTE 1: You should comment on your code wherever necessary and briefly explain what the code is doing.

NOTE 2: You need to encode the categorical columns, normalise the dataset, and remove the income column before applying the PCA method.

HINT: See the examples in the last three slides in Lecture #4 or the Lecture #4 exercise notebook

In [29]:
```
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

# Load dataset
data = pd.read_csv('https://raw.githubusercontent.com/maz786/WlvMScAI/mai

# Identifying numerical columns and categorical (excluding 'income' colum
categorical_cols = data.select_dtypes(include=['object']).columns.drop('i
numerical_cols = data.select_dtypes(include=['int64', 'float64']).columns

# Pipeline preprocessing creation
preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), numerical_cols),
        ('cat', OneHotEncoder(sparse_output=False), categorical_cols)  #
    ])

# Separating target variable and features
X = data.drop('income', axis=1)
y = data['income']

# Preprocessing Application
X_processed = preprocessor.fit_transform(X)

# PCA Application
```

```
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_processed)

# Converting the PCA result into a DataFrame for easier plotting
X_pca_df = pd.DataFrame(data=X_pca, columns=['Principal Component 1', 'Pr

# Plotting
plt.figure(figsize=(10, 8))
targets = ['<=50K', '>50K']
colors = ['r', 'b']
for target, color in zip(targets, colors):
    indicesToKeep = y == target
    plt.scatter(X_pca_df.loc[indicesToKeep, 'Principal Component 1'],
                X_pca_df.loc[indicesToKeep, 'Principal Component 2'], c=c
plt.title('PCA of Adult Income Dataset')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.legend(targets)
plt.show()
```



This code sample effectively preprocesses and visualises a dataset by using pandas for data processing and numpy for numerical operations (Wickham, 2014; Oliphant, 2006). In accordance with recommended techniques for data preprocessing, a ColumnTransformer integrates StandardScaler and OneHotEncoder for normalising numerical data and encoding categorical variables, respectively (Kuhn and Johnson, 2013). After preprocessing, principal component analysis (PCA) is used to split the dataset into two main components for visualisation. This method is recommended because it can effectively emphasise data variation (Abdi and Williams, 2010). The

matplotlib-created graphic illustrates the income categories present in the dataset and demonstrates how PCA may be used to clarify intricate data structures in real-world scenarios (Hunter, 2007). This methodology emphasises how important it is to incorporate reduction and preparation methods for data in order to enable thorough data analysis.

3D Interactive visualisations for Task 4.1 and 4.2

In [30]:
```
import pandas as pd
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
import plotly.graph_objects as go

# Step 1: Load  data - similar to opening a book to read it.
data = pd.read_csv('https://raw.githubusercontent.com/maz786/WlvMScAI/mai

# Task 4.1: Clustering and Visualization
# Choose 3 columns similar to picking 3 toys to play with.
columns_for_clustering = ['age', 'education-num', 'hours-per-week']
data_clustering = data[columns_for_clustering]

# Make the data nice and easy for  computer to understand - similar to so
scaler = StandardScaler()
data_clustering_scaled = scaler.fit_transform(data_clustering)

# Group the data into 3 clusters - similar to putting similar toys togeth
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
clusters = kmeans.fit_predict(data_clustering_scaled)

# Show the groups in a 3D space - similar to showing toys on a play mat.
fig1 = go.Figure(data=[go.Scatter3d(
    x=data_clustering_scaled[:, 0],  # Age
    y=data_clustering_scaled[:, 1],  # Education
    z=data_clustering_scaled[:, 2],  # Hours per week
    mode='markers',
    marker=dict(size=6, color=clusters, colorscale='Viridis', opacity=0.8
)])
fig1.update_layout(title="Task 4.1: 3D Clusters Visualization", margin=di
fig1.show()

# Task 4.2: PCA and its Visualization
# Prepare the data, but keep 'income' out of it - similar to preparing a
features = data.drop('income', axis=1)
numeric_features = features.select_dtypes(include=['int64', 'float64']).c
categorical_features = features.select_dtypes(include=['object']).columns

# Make sure everything is understandable for the computer, including word
preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), numeric_features),
        ('cat', OneHotEncoder(sparse_output=False), categorical_features)
    ]
)

features_processed = preprocessor.fit_transform(features)
```

```
# Look at the data in a special way, showing its shape but in just 3 aspe
pca = PCA(n_components=3)
principal_components = pca.fit_transform(features_processed)

# Color the points by whether people make more or less money - similar to
income_colors = data['income'].apply(lambda x: 1 if x == '>50K' else 0).v

# Show this special view in 3D space - similar to looking at a 3D movie.
fig2 = go.Figure(data=[go.Scatter3d(
    x=principal_components[:, 0],  # Shadow aspect 1
    y=principal_components[:, 1],  # Shadow aspect 2
    z=principal_components[:, 2],  # Shadow aspect 3
    mode='markers',
    marker=dict(size=6, color=income_colors, colorscale='Viridis', opacit
)])
fig2.update_layout(title="Task 4.2: 3D PCA Visualization by Income", marg
fig2.show()
```

This code snippet illustrates how pandas may be used to efficiently handle tabular datasets by importing and manipulating data (Reback et al., 2020). In order to prepare data for machine learning tasks, it uses StandardScaler and OneHotEncoder from scikit-learn for data preprocessing. This allows for the standardisation of numerical data and the encoding of categorical variables in a machine-readable format (Buitinck et al., 2013). KMeans, a technique for performing clustering, divides the data into k different groups according to feature similarity (Arthur & Vassilvitskii, 2007). The algorithm then reduces the dataset to three principal components using PCA from scikit-learn for dimensionality reduction. This helps with visualisation and may even enhance clustering performance by capturing the most significant variance in the data (Jolliffe & Cadima, 2016). Plotly.graph_objects is a package that makes interactive and visually appealing plots for web applications. It is used to visualise clusters and PCA components (Plotly Technologies Inc., 2015). This makes it possible to represent clusters and PCA-reduced data in three dimensions, giving rise to a clear comprehension of variance and data grouping.

Bibliography

Abdi, H. and Williams, L.J., 2010. Principal component analysis. Wiley Interdisciplinary Reviews: Computational Statistics, 2(4), pp.433-459.

Arthur, D. and Vassilvitskii, S., 2007. k-means++: The advantages of careful seeding. In: Proceedings of the eighteenth annual ACM-SIAM symposium on Discrete algorithms. Society for Industrial and Applied Mathematics, pp.1027-1035.

Buitinck, L., Louppe, G., Blondel, M., Pedregosa, F., Mueller, A., Grisel, O., Niculae, V., Prettenhofer, P., Gramfort, A., Grobler, J., Layton, R., VanderPlas, J., Joly, A., Holt, B., and Varoquaux, G., 2013. API design for machine learning software: experiences from the scikit-learn project. arXiv preprint arXiv:1309.0238.

Hunter, J.D., 2007. Matplotlib: A 2D graphics environment. Computing in Science & Engineering, 9(3), pp.90-95.

Johnson, S.C., 1967. Hierarchical clustering schemes. Psychometrika, 32(3), pp.241-254.

Jones, E., Oliphant, T., Peterson, P., et al., 2001. SciPy: Open source scientific tools for Python.

Jolliffe, I.T., 2002. Principal component analysis for special types of data. Principal component analysis, 2, pp.487-501.

Jolliffe, I.T. and Cadima, J., 2016. Principal component analysis: a review and recent developments. Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences, 374(2065), 20150202.

Kodinariya, T.M. and Makwana, P.R., 2013. Review on determining the number of Clusters in K-Means Clustering. International Journal, 1(6), pp.90-95.

Kuhn, M. and Johnson, K., 2013. Applied Predictive Modeling. New York: Springer.

MacQueen, J., 1967. Some methods for classification and analysis of multivariate observations. Proceedings of the 5th Berkeley Symposium on Mathematical Statistics and Probability, 1(14), pp.281-297.

McKinney, W., 2010. Data structures for statistical computing in python. Proceedings of the 9th Python in Science Conference, 445, pp.51-56.

Oliphant, T.E., 2006. A guide to NumPy. USA: Trelgol Publishing.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., and Duchesnay, E., 2011. Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, pp.2825-2830.

Plotly Technologies Inc., 2015. Plotly.

Reback, J., McKinney, W., jbrockmendel, Van den Bossche, J., Augspurger, T., Cloud, P., Hawkins, S., 2020. pandas-dev/pandas: Pandas. Zenodo.

Van Der Walt, S., Colbert, S.C., and Varoquaux, G., 2011. The NumPy array: A structure for efficient numerical computation. Computing in Science & Engineering, 13(2), pp.22-30.

Wickham, H., 2014. Tidy Data. Journal of Statistical Software, 59(10), pp.1-23.

############# WRITE YOUR REPORT IN THIS CELL (IF APPLICABLE)####################